

Real-Time Taxi/Transportation Telemetry Streaming Pipeline

Project Overview

Objective:

Build a scalable real-time streaming data solution to ingest, enrich, store, analyze, and visualize NYC taxi trip telemetry events, supporting live dashboards, micro-billing, and cost-effective retention.

Primary Stakeholders:

- Data Engineers
 - Data Analysts
 - Operations/Business Team
 - End Users (Web dashboard viewers)
-

Business Scenario & Goals

- **Scenario:**
Enable real-time ingestion and processing of taxi GPS and trip events for analytics, live dashboards, and accurate billing.
 - **Business Goals:**
 - Provide always up-to-date taxi trip analytics, utilization, and metrics
 - Support instant reporting and insights for resource optimization
 - Simplify billing and auditing (micro-billing per trip)
 - Maintain cost-optimized and secure data storage and retention
 - Ensure platform is robust, resilient and easily monitored
-

Data Sources

- **Trip Events Dataset:**
 - NYC Taxi Trip Records (public Kaggle dataset)
 - Sample rate: ~1000 events/sec (simulation)
 - Fields: Taxi ID, Pickup/Dropoff times, locations, trip distance, fare details, payment type, etc.
 - **Zone Lookup Table:**
 - TLC Taxi Zone Lookup (LocationID, Borough, Zone, service_zone)
-

Functional Requirements

#	Requirement
1	Ingest GPS/trip event data at ~1000 events/sec using Amazon Kinesis Data Streams
2	Enrich trip records with zone metadata via Lambda (uses DynamoDB zone lookup)
3	Persist raw/minimally processed trip records to Amazon S3 (Parquet, partitioned by date/hour)
4	Store cleaned/enriched event streams in Amazon Redshift (batch via Kinesis Firehose)
5	Micro-billing: For every trip, calculate and store per-trip fare/timestamp in DynamoDB for low-latency, fast queries
6	Provide business and operational dashboards via AWS QuickSight (Redshift and DynamoDB as data sources)
7	Offer a live web map visualization (web UI using serverless API Gateway + Lambda)
8	Implement monitoring: CloudWatch metrics and alarms for: • Data lag/delivery issues • Throttling (Kinesis, Firehose, Lambda) • Failures at every pipeline stage 9 Data retention and cost controls (S3 lifecycle rules to IA/Glacier; Kinesis, DynamoDB TTL, and Redshift compression/unload/archive) 10 Security: IAM least privilege, encryption at rest (KMS/SSE), API key or Cognito authentication, VPC isolation for Lambdas

Data Pipeline Architecture

Ingestion:

- Kinesis Data Streams (partition keys: taxi_id for load distribution)
- Simulated sender produces events at ~1000/sec

Processing/Enrichment:

- Lambda triggers off Kinesis stream
- Fetches zone info from DynamoDB
- Enriches trip event with location context (Borough/Zone)

Storage:

- Raw: Kinesis Firehose to S3
- Cleaned: Kinesis Firehose to Redshift (schema-aligned, micro-batches)
- Billing: DynamoDB table (Primary key: trip_id, Sort key: timestamp; GSI for taxi queries)

Analytics & Visualization:

- QuickSight dashboards for monitoring and KPIs

Monitoring & Alerts:

- CloudWatch metrics for **datalag**, **delivery failure** and alarms and notification sent to email stream throttling(natively available in AWS)

Retention/Cost Optimization:

- S3: Lifecycle to IA/Glacier, expiry
- Kinesis/DynamoDB: retention/TTL
- Redshift: compression, unload/archive older data

7. Implementation Plan

- Infrastructure-as-Code: **AWS CDK (Python)**
- GitHub for version control
- Stepwise construction: Build,validate each stage one at a time

End-to-End Pipeline Flow:

Producer (simulated events):

- **What it is:** In our taxi example, these would be the individual taxi vehicles generating telemetry events. "Simulated events" suggests that for development or testing, you might be using a program to generate realistic-looking data rather than real devices.
- **Role:** Generates and sends a continuous stream of raw data points or "events."

2. Kinesis Data Stream (Ingest):

- **What it is:** AWS Kinesis Data Streams is a highly scalable and durable real-time data streaming service. It acts as a buffer and a transport layer for streaming data.
- **Role:** Ingests raw events from the Producers. It ensures that the data is collected reliably, can handle massive volumes, and provides a persistent, ordered log of events that multiple consumers can read from independently. It's the central nervous system for your real-time data.

3. Lambda (Enriches):

- **What it is:** AWS Lambda is a serverless compute service that runs code in response to events. In this diagram, it's triggered by new records arriving in the Kinesis Data Stream.
- **Role:** This is where your real-time processing and *enrichment* logic resides. When new data arrives in Kinesis, Lambda wakes up, reads the data, and performs operations like:
 - **Validation:** Checking if the data conforms to expected formats.
 - **Transformation:** Changing data types, reformatting.
 - **Enrichment:** Adding contextual information. For example, if a telemetry event only contains a latitude/longitude, Lambda might query DynamoDB (as shown below) to find out which NYC borough or pricing zone that coordinate falls into, and then add that information to the event.
 - **Filtering:** Dropping irrelevant events.

4. DynamoDB (zone & billing):

- **What it is:** AWS DynamoDB is a fast, flexible NoSQL database service for single-digit millisecond performance at any scale.
- **Role (in relation to Lambda):** This is depicted as a *reference data store*. Lambda queries DynamoDB to retrieve information needed to enrich the streaming data. For instance:
 - **Zone Information:** Looking up geographical zone details (e.g., "Is this GPS coordinate in a surge pricing zone?").
 - **Billing Rules:** Fetching dynamic pricing rules, driver commission rates, or toll schedules that Lambda applies during enrichment for micro-billing purposes.

5. Firehose Streams (raw/cleaned):

- **What it is:** Amazon Kinesis Firehose is a fully managed service for delivering real-time streaming data to destinations like Amazon S3, Amazon Redshift, Amazon OpenSearch Service, and Splunk. It automatically scales to match the throughput of your data and requires no ongoing administration.
- **Role:** After Lambda has processed and enriched the data, it sends the *transformed* data to Firehose. Firehose then efficiently delivers this data to its configured destinations. The "raw/cleaned" annotation suggests that Firehose might be receiving both the original raw data (perhaps from a different path in Kinesis, or directly from Lambda for archival) and the cleaned/enriched data, or that it's delivering the *cleaned* version of the *raw* stream to various targets. It acts as an easy way to persist streaming data without writing custom consumers.

6. S3 / Redshift / DynamoDB:

- **What it is:** These are common AWS data storage services, serving different purposes.
 - **S3 (Simple Storage Service):** Object storage, highly durable and cost-effective, ideal for data lakes, raw data archives, and historical data.

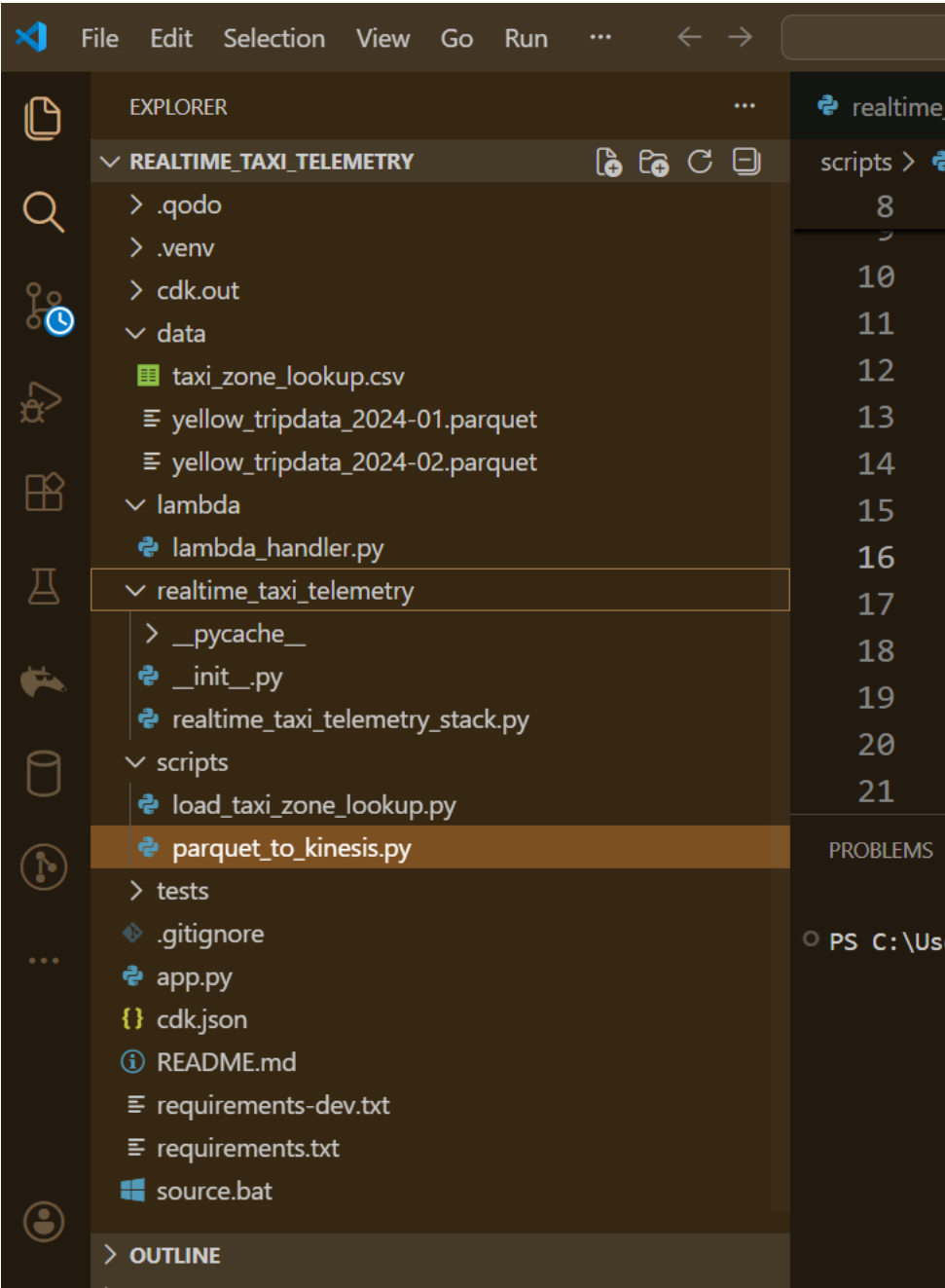
- **Redshift:** A fully managed, petabyte-scale cloud data warehouse service, optimized for analytical queries over large datasets.
- **DynamoDB:** (Again) A NoSQL database, also suitable for storing processed events that require low-latency access for specific applications (e.g., micro-billing records, live state for dashboards).
- **Role:** These are the *destinations* for the processed and enriched streaming data delivered by Firehose.
 - **S3:** Likely stores all raw and/or processed events for long-term archiving, compliance, or as a data lake for future analytics and machine learning.
 - **Redshift:** Would store the cleaned and enriched data, potentially denormalized, for complex analytical queries, reporting, and business intelligence. This would be our "Batch Analytics / Data Warehouse."
 - **DynamoDB:** Could store specific aggregate metrics or final trip records for the micro-billing application to quickly look up or for displaying on a live dashboard that needs very low latency reads. This would be part of our "Hot Data Store" or for specific application lookups.

7. CloudWatch Monitoring/Alarms (lag/throttle/failure, usage, etc):

- **What it is:** AWS CloudWatch is a monitoring and observability service that provides data and actionable insights for AWS, hybrid, and on-premises applications and resources.
 - **Role:** This is critical for operational excellence. It monitors the health and performance of all the AWS services in the pipeline (Kinesis, Lambda, Firehose, DynamoDB). It collects metrics (e.g., Kinesis shard utilization, Lambda invocations/errors/duration, DynamoDB consumed capacity) and allows you to set up alarms.
 - **Why:** Alerts on issues like:
 - **Lag:** If Lambda is falling behind in processing Kinesis records.
 - **Throttling:** If services are unable to handle the incoming data rate.
 - **Failures:** If Lambda functions are erroring out.
 - **Usage:** Monitoring resource consumption for cost optimization.
-

IMPLEMENTATION:

Project Folder Structure



Python Virtual Environment

Command powershell

```
python -m venv .venv  
.\venv\Scripts\Activate.ps1
```

Why:

Isolates project dependencies from your system Python, making environment reproducible and avoiding conflicts.

Installing Dependencies

Dependencies:

- `aws-cdk-lib`: AWS Cloud Development Kit core library for infrastructure-as-code.
- `constructs`: Base CDK constructs dependency.
- `boto3`: AWS SDK for Python for interacting with AWS APIs (scripted tasks).
- `pandas`: For CSV/data processing.

Command:

```
powershell  
pip install aws-cdk-lib constructs boto3 pandas
```

Why:

- CDK libraries are needed for deploying cloud infrastructure in a repeatable way.
- `boto3` and `pandas` enable data operations (like CSV load) from Python scripts.

Infrastructure Setup with AWS CDK:

Code for creating Infra stack:

https://github.com/vamshisales11/realtime_taxi_telemetry/blob/main/realtime_taxi_telemetry_stack.py

RealtimeTaxiTelemetryStack:

Infrastructure Overview and Code Explanation:

This AWS CDK stack provisions the core infrastructure required for ingesting, processing, storing, and analyzing taxi telemetry data in a near-real-time, cloud-native fashion.

Kinesis Data Stream: TaxiTripEventStream

A Kinesis stream is defined to ingest all incoming trip telemetry events.

- The stream is named `TaxiTripEventStream` and uses two shards for parallel ingestion.
- The stream is provisioned for fixed capacity, supporting high-throughput use cases.
- Retention is set for 24 hours to allow downstream consumers sufficient time for processing.

DynamoDB Tables

Two DynamoDB tables are provisioned:

1. **TaxiZoneLookup Table**
 - Stores static reference data mapping each taxi zone's location ID to its borough and zone name.
 - Uses numeric `LocationID` as the partition key for efficient lookups.
 - Removal policy is set to destroy for rapid stack teardown during development.
 - Billing is set to pay-per-request for cost flexibility.
2. **TaxiMicroBilling Table**
 - Records individual trip billing transactions ("micro-billing").
 - Uses `trip_id` as the partition key and `timestamp` as the sort key to support time-series access patterns.
 - Time-to-live attribute (`ttl`) is enabled for automatic cleanup of obsolete records.
 - Removal policy and billing mirror the zone table for ease of management.

S3 Bucket: Raw Taxi Event Storage

An S3 bucket, `taxi-raw-event-persist001`, is created to retain all raw taxi telemetry events with the following configurations:

- No versioning to minimize storage overhead.
- Lifecycle rules transition data to lower-cost storage classes: Glacier after 30 days, Intelligent Tiering after 7 days.
- All objects are set to expire after 365 days for cost control.
- Removal policy is destroy, simplifying non-production stack deletions.

IAM Roles: Firehose Delivery

The stack provisions a dedicated IAM role for Amazon Kinesis Data Firehose (`FirehoseDeliveryRole`), enabling Firehose to access the Kinesis stream and S3 bucket for delivery operations.

- The role grants write access to S3 and the necessary Kinesis actions (`DescribeStream`, `GetShardIterator`, `GetRecords`, `ListShards`) on the specific event stream resource.

Firehose Delivery Stream: Raw Events

A Kinesis Data Firehose delivery stream, `taxi-raw-events-stream`, connects the Kinesis stream to the S3 bucket:

- The Firehose delivery stream receives events, buffers data to control batching (max 5 MB or 60 seconds), and writes batches to S3.
- Objects are partitioned and stored with dynamic prefixes based on event timestamp (month, day, hour).
- Failed records are routed to a dedicated S3 path (`raw_events/errors/`).

Lambda Function: Event Enrichment

A Lambda function, `TaxiEnrichmentLambda`, performs real-time event enrichment and downstream delivery:

- Python 3.10 runtime is used with a main handler in the `lambda` directory.
- Function timeout is set for five minutes to accommodate longer batch processing.
- Lambda is configured to read from DynamoDB's zone table for reference lookups.
- Lambda writes billing records to the micro-billing table and can send enriched events to a second Firehose stream named `taxi-clean-events-stream` using explicit role policy permissions.

Kinesis-Lambda Event Source Wiring

The Lambda function is connected to the Kinesis stream as an event source:

- The function processes events in batches of 100, starting from the latest stream position, allowing scalable real-time enrichment.

Redshift Serverless Configuration

The stack provisions the following Redshift serverless resources:

1. **Namespace:** `taxi-namespace`
 - Provides a logical Redshift environment, with a dedicated admin user/password.
2. **Workgroup:** `taxi-workgroup`
 - Implements compute resources to run analytic workloads, public accessibility enabled, and base capacity set to eight units.
 - Workgroup creation depends on namespace provisioning to ensure correct resource hierarchy.

Secrets Manager: Redshift Credentials

A Secrets Manager secret, `TaxiRedshiftAdminCreds`, securely stores Redshift admin credentials:

- The secret is generated with a template containing the username and a randomly created password with no punctuation, ensuring compliance with requirements.
- Password length is set to sixteen for security.

IAM Role: Firehose Redshift Delivery

A second IAM role, `FirehoseRedshiftDeliveryRole`, is created:

- This role enables Firehose delivery to Redshift.
- The `AmazonRedshiftFullAccess` managed policy is attached to allow creation, modification, and loading operations in Redshift.
- The role is also granted read access to the S3 bucket, necessary for staging data before Redshift ingestion.

Bulk-Loading Zone Lookup Data

CSV Data:

`data/taxi_zone_lookup.csv` contains mappings like:

LocationID	Borough	Zone	service_zone
264	Unknown	N/A	N/A
265	N/A	Outside of NYC	N/A

Load Script (in `scripts/load_taxi_zone_lookup.py`):

https://github.com/vamshisales11/realtime_taxi_telemetry/blob/main/scripts/load_taxi_zone_lookup.py

Code Explanation:

- The script reads a CSV (`taxi_zone_lookup.csv`) containing taxi zone definitions using pandas.
- For each record, it prepares a structured dictionary (item) compatible with DynamoDB, ensuring missing fields are filled with “N/A” and location IDs are properly validated.
- Entries with an invalid LocationID are skipped to maintain database integrity.
- Each valid taxi zone is inserted into the `TaxiZoneLookup` DynamoDB table, suitable for rapid lookups as part of a data pipeline.
- This operation enables mapping zone IDs to their details in downstream analytics or processing steps by storing the reference data in DynamoDB, which is scalable and fast for key-value lookups.
- The print statements provide progress and error tracking during execution.

This script is typically used in ETL pipelines for loading reference data, ensuring reliable zone metadata lookup for each taxi trip processed..

After writing Code: Deploy it using “CDK Synth followed by CDK Deploy”

InfraStructure Screenshots:

Data streams (1)

Info

Process data in real time

Create

Find data streams

Name

Status

Capacity mode

Maximum record size

[TaxiTripEventStream](#)

Active

Provisioned

1024 KiB

Firehose streams (2)

Info

You can create a Firehose stream to set up a source, destination, and optional transformation for your streaming data delivery.

Find Firehose streams

Name

Status

Creation time

Source

[taxi-clean-events-stream](#)

Active

November 16, 20...

Direct PUT

[taxi-raw-events-stream](#)

Active

November 17, 20...

[TaxiTripEventStre...](#)

DynamoDB

Tables

DynamoDB

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations

Tables (2)

Info

Find tables

Filter by tag

Any tag key

Filter by tag value

Any tag value

Name

Status

Partition key

Sort key

Indexes

Replication Regions

Deletion

[TaxiMicroBilling](#)

Active

trip_id (S)

timestamp (S)

0

0

Off

[TaxiZoneLookup](#)

Active

LocationID (N)

-

0

0

Off

Lambda

Functions

Lambda

Dashboard

Applications

Functions

Additional resources

Code signing configurations

Event source mappings

Layers

Replicas

Functions (1)

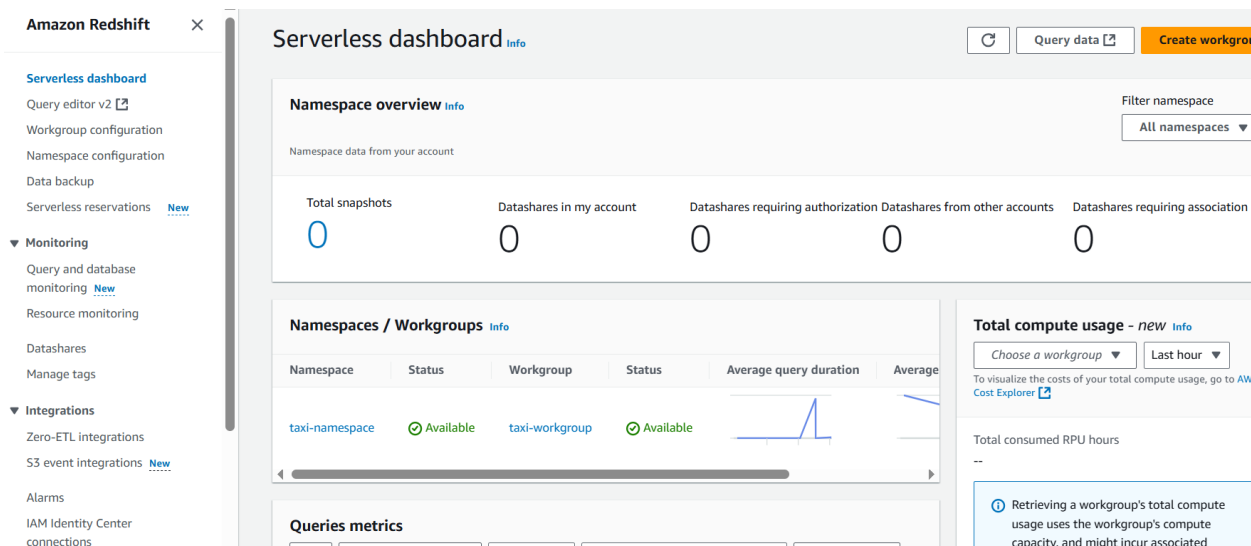
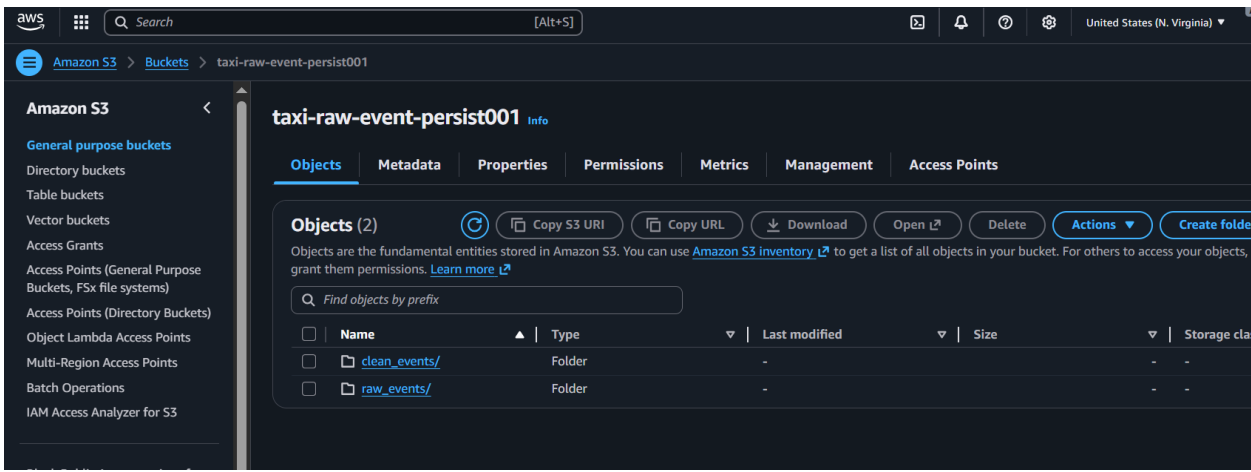
Search by attributes or search by keyword

Function name

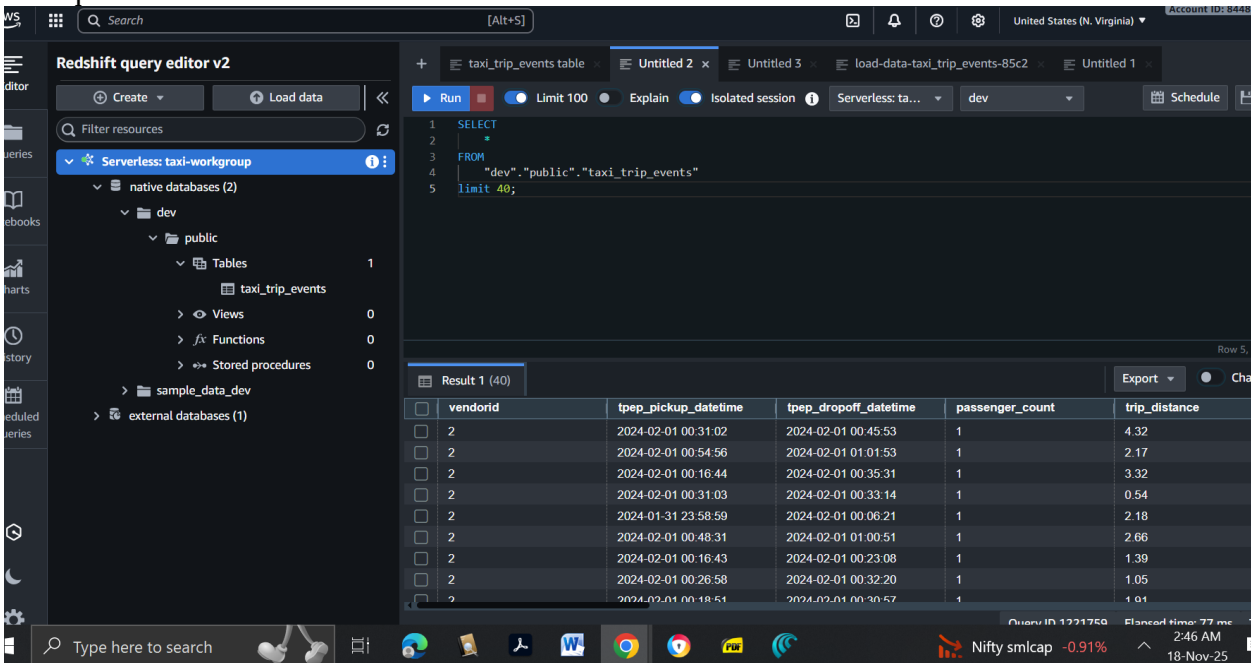
Description

[RealtimeTaxiTelemetryStac-TaxiEnrichmentLambda8C5F-fERX4FqiXAxQ](#)

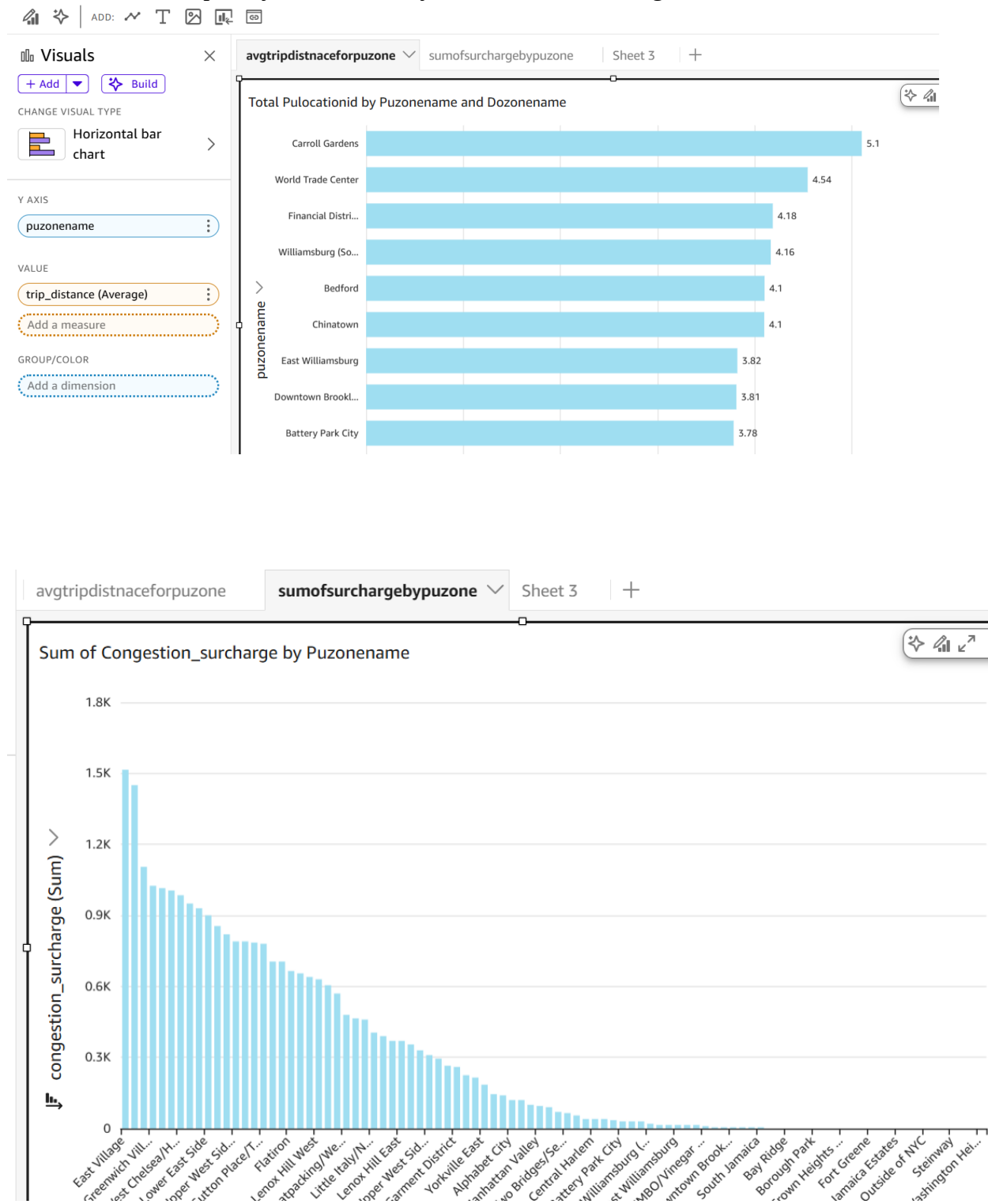
-

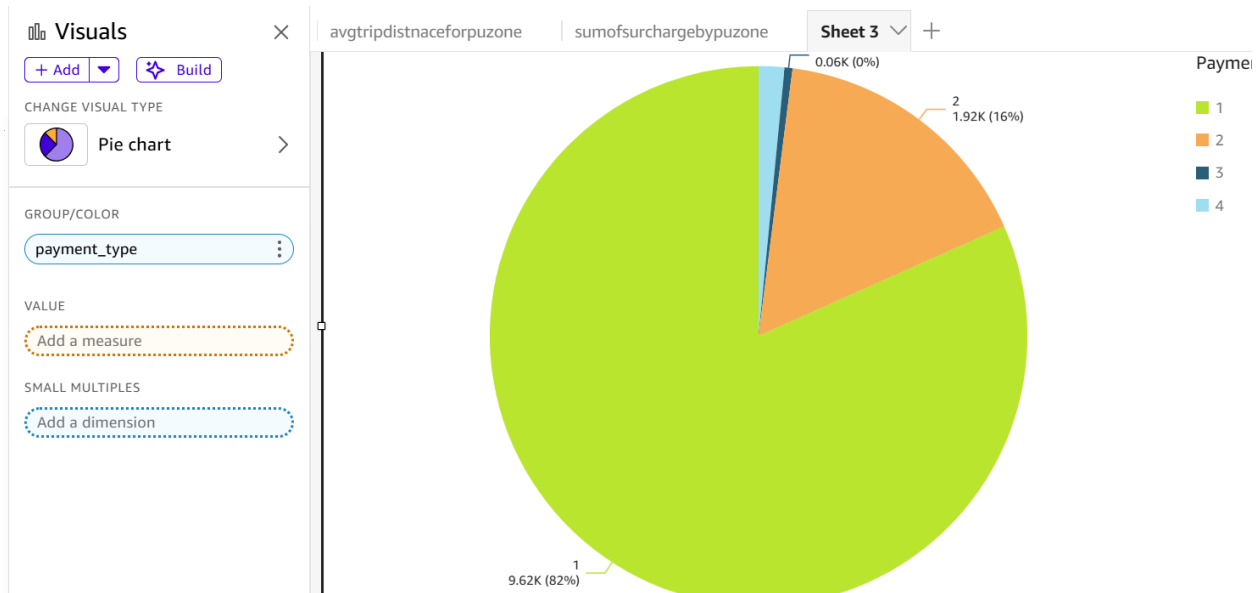


data queried into Redshift:



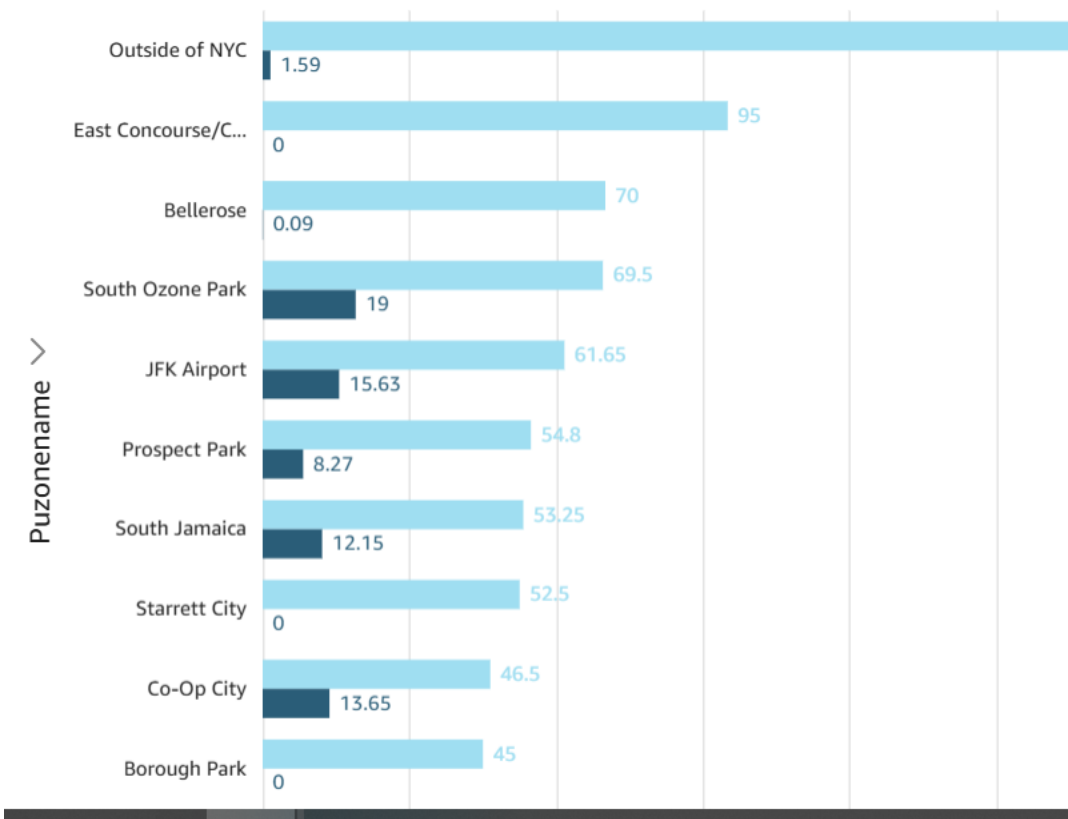
Named them descriptively to make it easy to understand: could go with further visualization and analysis



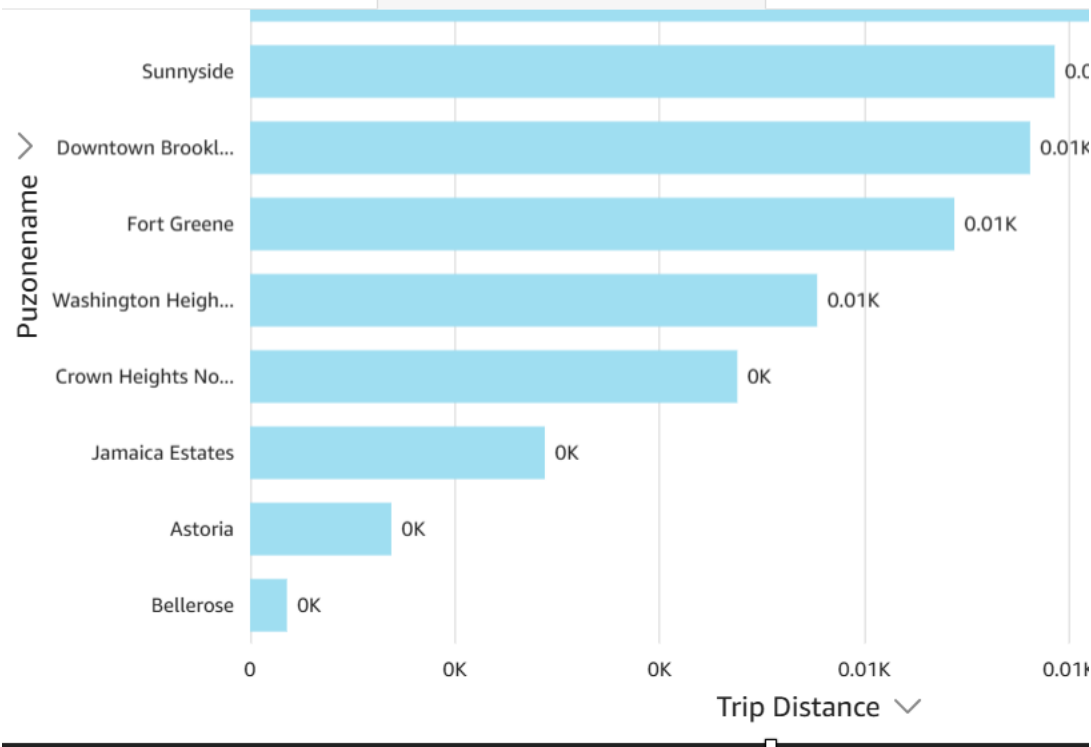


Average of Fare Amount and average of Trip Distance by Puzonename:

Average of Fare Amount and average of Trip Distance by Puzonename



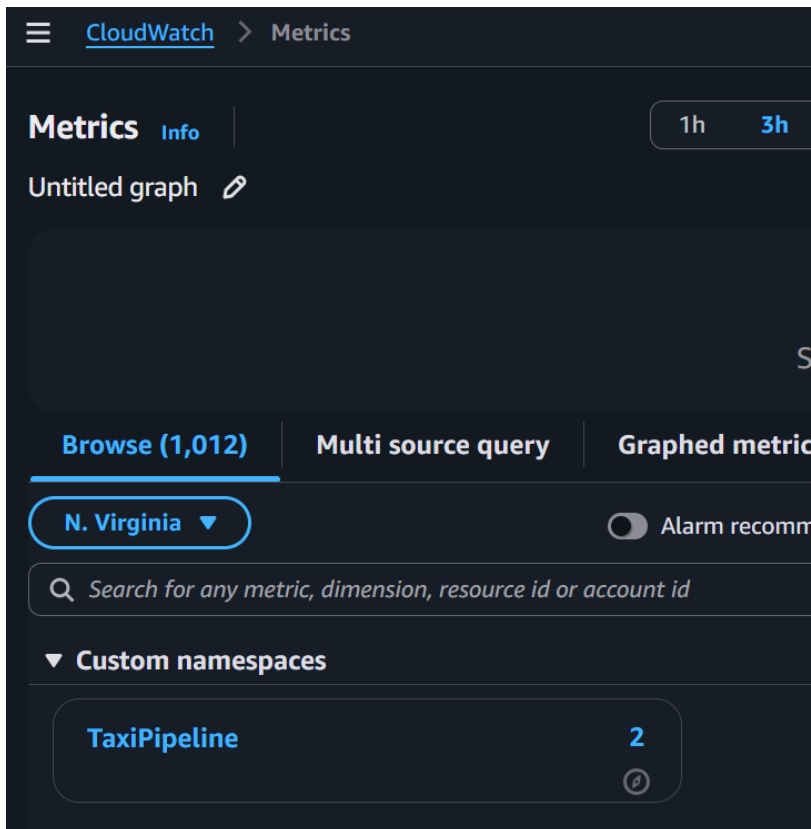
Total Trip Distance by Puzonename.



Monitoring:

CloudWatch metrics + alarms for data lag, stream throttling, and delivery failures.

We create a TaxiPipeline namespace in lambda for metrics:



- **Data Lag:**
Measures delay between actual event time (`tpep_pickup_datetime`) and processing in Lambda.
 - **Importance:** High lag indicates pipeline slowdowns or bottlenecks.
 - **Action:** Lambda emits lag metric; CloudWatch alarm triggers if lag > 5 minutes (300s).
- **Delivery Failure:**
Tracks counts of record failures during validation, enrichment, database write, or delivery (Firehose/Redshift).
 - **Importance:** Ensures reliability; zero tolerance for data loss.

Action: Lambda emits failure count; CloudWatch alarm activates upon any detected failure

Browse (2)

Multi source query

Graphed metrics

Options

Source

N. Virginia

Q

Search for any metric, dimension, resource id or account id

Stream 2/2

Metric name

Alarms

taxi-clean-events-stream

DataLagSeconds

No alarms

taxi-clean-events-stream

DeliveryFailure

No alarms

Browse (2)

Multi source query

Graphed metrics (2)

Options

Source

Add math

Add query

Add dynamic label

Info

Average

5 minutes

Clear graph

Label	Details	Statistic	Period	Y axis	Actions
DataLagSeconds	TaxiPipeline • DataLagSeconds • Stream: TaxiPipeline • DataLagSeconds • Stream: taxi-clean-events-stream	Average	5 minu...	< >	Refresh Alert Copy Share Close
DeliveryFailure	TaxiPipeline • DeliveryFailure • Stream: TaxiPipeline • DeliveryFailure • Stream: taxi-clean-events-stream	Average	5 minu...	< >	Refresh Alert Copy Share Close

Alarm Set:

Alarms

Datalagalarm

Actions

View

Explore related

Investigate

Details

Name

Datalagalarm

Type

Metric alarm

Description

No description

State

Insufficient data

Threshold

DataLagSeconds > 300 for 1 datapoints within 5 minutes

Actions

Actions enabled

Last state update

2025-11-18 02:18:55 (Local)

Namespace

TaxiPipeline

Metric name

DataLagSeconds

Stream

taxi-clean-events-stream

Statistic

Average

Period

5 minutes

Datapoints to alarm

1 out of 1

Missing data treatment

Treat missing data as missing

Percentiles with low samples

evaluate

ARN

arn:aws:cloudwatch:us-east-1:844840482726:alarm:Datalagalarm

View EventBridge rule

- **Stream Throttling (Native AWS Metric):**

Occurs when Kinesis write/read rates exceed shard limits (`PutRecords.Throttled`, `GetRecords.Throttled`).

- **Importance:** Throttling can delay or drop data, affecting real-time delivery.
- **Action:** CloudWatch alarm if throttling count > 0; prompts investigation or scaling.

- **Dashboards and Alerts:**

All metrics and alarms are visualized in CloudWatch. Alarm notifications (email) ensure prompt visibility and response.

These monitoring tools provide instant awareness of pipeline delays, data loss, and throughput bottlenecks—enabling proactive maintenance and reliability for AWS streaming data workflows.

Conclusion:

The TaxiTelemetryProject achieves end-to-end automation across cloud infrastructure provisioning, streaming data ingestion, processing, and storage. Manual intervention is limited to running the initial producer code and configuring visualizations in QuickSight. This balanced approach ensures rapid deployment and operational efficiency while providing the flexibility for custom data input and tailored analytics. The architecture is extensible, supporting further automation and scalability as project needs evolve.